Sesión 1

Fundamentos Técnicos y Estratégicos

Alberto Cámara · ISDI MDA · 2026

Sobre el Instructor

Alberto Cámara

- PhD en Matemáticas, Freelancer
- Experiencia liderando equipos de datos
- Organizador en **PyBCN**
- Frecuentemente doy clases de **Data Analysis, Machine Learning y más**

<> ¿Quiénes sois?

- ¿Con qué frecuencia usáis ChatGPT, Copilot u otra IA generativa?
- ¿Habéis llamado alguna vez a una API de LLM desde código?
- ¿Tenéis experiencia con Python / notebooks?
- ¿Qué esperáis llevarnos de estas 9 horas?

<> Cómo vamos a trabajar

- Interrumpid cuando algo no quede claro — no hay preguntas tontas
- Mezclaremos teoría y demo en directo en cada bloque
- Los notebooks son vuestros: experimentad, romped cosas
- Si algo no funciona en Colab, lo resolvemos juntos

<> Agenda — Sesión 1

Bloque 1 (80 min)

- Demo de apertura: ¿qué puede hacer esto?
- Del ML clásico a los Foundation Models
- Arquitectura Transformer (alto nivel)
- LLMs principales
- Embeddings y espacios vectoriales
- Demo en vivo: primera llamada a Gemini API

Bloque 2 (80 min)

- Fine-tuning vs. prompting vs. RAG
- Ecosistema actual: cloud vs. local
- ⚠️ **Sección crítica**: el coste real de la IA
- Riesgos y limitaciones
- Práctica guiada

<> Bloque 1

Qué es esto y cómo funciona

<> ¿Qué puede hacer esto?

Tenemos esta review de Amazon Electronics:

```
"Bought this headset for my home office. The noise cancellation is
incredible — I can focus even when my kids are running around.
Battery lasts 30+ hours. Only downside: the earcups get hot after
2-3 hours. Worth every penny."
```

Vamos a pedirle al modelo que:

1. **Resuma** esta review en una frase en español
2. **La clasifique** como positiva, negativa o mixta
3. **Proponga una respuesta** de atención al cliente

<> Del ML Clásico a los Foundation Models

ML tradicional (años 90–2010)

- Modelos entrenados para **una tarea específica**
- Regresión logística, SVM, Random Forest
- Feature engineering manual
- Requieren datos etiquetados abundantes

Deep Learning (2010–2017)

- Redes neuronales profundas
- Aprenden representaciones automáticamente
- Dominan en visión (CNN) y texto (RNN/LSTM)

<> Del ML Clásico a los Foundation Models

Foundation Models (2017–hoy)

- Entrenados en **cantidades masivas** de datos
- Un solo modelo → múltiples tareas sin reentrenamiento
- Capacidades **emergentes** no programadas
- Transferencia de conocimiento generalizada

El salto no es solo cuantitativo (más datos, más parámetros): es cualitativo. Aparecen capacidades que nadie diseñó explícitamente.

<> Capacidades Emergentes

¿Qué son?

Habilidades que aparecen de forma abrupta al superar cierto umbral de escala — no se observan en modelos más pequeños.

Ejemplos documentados

- Razonamiento en cadena (chain-of-thought)
- Aritmética de varios pasos sin ejemplos previos
- Traducción zero-shot a idiomas minoritarios
- Comprensión de humor y sarcasmo
- Generación de código funcional

<> Capacidades Emergentes

¿Por qué importa?

Los modelos grandes hacen cosas que **nadie programó** y que nadie predijo a partir de los modelos pequeños.

Esto también los hace **impredecibles** y difíciles de auditar. El mismo mecanismo que produce razonamiento útil produce alucinaciones convincentes.

<> Arquitectura Transformer (alto nivel)

El problema que resuelve

El texto tiene dependencias a larga distancia:

> *"El banco que estaba junto al río [...20 palabras...] se desbordó"*

Las RNNs "olvidan" lo lejano. Los Transformers, no.

Mecanismo de atención

Cada token puede "prestar atención" a cualquier otro token del contexto, sin importar la distancia entre ellos.

<> Arquitectura Transformer — Conceptos clave

Concepto	Analogía simple
Token	Fragmento de texto (~¾ de palabra)
Contexto	Ventana de memoria del modelo
Atención	Peso de relevancia entre tokens
Embedding	Posición en espacio semántico

<> Los LLMs No "Leen": Predicen

Lo que hace realmente el modelo

Dado un texto de entrada, calcula una **distribución de probabilidad** sobre el siguiente token:

> *"La capital de Francia es ___"*
>
> `París` *: 94.2%*
>
> `Lyon` *: 1.1%*
>
> `una` *: 0.8%*

Genera token a token, usando lo que ya generó como nuevo contexto.

<> Los LLMs No "Leen": Predicen

Consecuencia clave

El modelo no "sabe" que París es la capital de Francia en el sentido humano.

Aprendió que después de "La capital de Francia es", el token "París" aparece frecuentemente en el corpus de entrenamiento.

Esta distinción importa para entender las **alucinaciones**.

<> LLMs Principales en Abril de 2026

Proveedor	Modelo	Tipo	Contexto	Puntos fuertes
OpenAI	GPT-5	Cerrado	1M	Multimodal, general
Anthropic	Claude Opus 4.7	Cerrado	1M	Razonamiento, instrucciones
Google	Gemini 3.1 Pro	Cerrado	1M	Contexto largo, integración Google
Meta	Llama 4	**Abierto**	10M	Ejecutable localmente **(*)**
Mistral	Mistral Large	Abierto	256K	Europa, eficiencia
Alibaba	Qwen 3.6	**Abierto**	1M	Multilingüe, muy eficiente

<> Modelos abiertos vs. cerrados

Los modelos **abiertos** (Llama, Mistral) pueden ejecutarse en hardware propio — control total de datos.

Los **cerrados** ofrecen más capacidad pero implican dependencia de proveedor.

<> Embeddings: Representación Semántica

Un embedding convierte texto en un vector de números

> *"Este auricular tiene buena cancelación de ruido" → [0.23, −0.11, 0.87, ...] (768 dimensiones)*

¿Para qué sirven?

- Medir **similitud semántica** entre textos
- Buscar documentos relevantes (base del RAG)
- Clustering de reseñas o documentos
- Clasificación sin etiquetas

Distancia = similitud

<> Embeddings: Ejemplo concreto

```
"batería de larga duración"
≈ "autonomía de 30 horas"
  (cosine similarity: 0.91)

"batería de larga duración"
≉ "entrega rápida"
  (cosine similarity: 0.23)
```

La similitud se calcula sin buscar palabras exactas.

<> Demo: Proyección 2D de Reviews

¿Qué pasa si tomamos 50 reviews de electrónica y proyectamos sus embeddings a 2 dimensiones?

Proceso

1. Generar embedding para cada review (768 dims)
2. Reducir a 2D con PCA
3. Colorear por rating (1–5 estrellas)

Lo que esperamos ver

- Reviews positivas (4–5★) agrupadas
- Reviews negativas (1–2★) agrupadas
- Separación no perfecta, pero sí tendencia

<> Demo: Proyección 2D — Por qué importa

- Revela estructura semántica en los datos
- Fundamento visual del por qué funciona la búsqueda vectorial
- Intuición necesaria para entender RAG (Sesión 2)

→ *Demo en notebook*

<> Demo en Vivo: Primera Llamada a Gemini API

```
# 1. Instalar
%pip install -q google-genai pandas

# 2. Configurar (API key en Colab Secrets)
from google import genai
from google.colab import userdata

client = genai.Client(api_key=userdata.get('GOOGLE_API_KEY'))
MODEL = 'gemini-2.5-flash'

# 3. Primera llamada
review = "Bought this headset for my home office..."
```

22

```
response = client.models.generate_content(
```

<> Descanso

15 minutos

<> Bloque 2

Estrategia, Ecosistema y Riesgos

<> Fine-tuning vs. Prompting vs. RAG

Estrategia	Cuándo usarla	Coste	Dificultad
Prompting	La mayoría de los casos	Bajo	Baja
RAG	Cuando el modelo necesita información actualizada o privada	Medio	Media
Fine-tuning	Estilo muy específico o dominio muy técnico	Alto	Alta

<> Fine-tuning vs. Prompting vs. RAG — Regla práctica

Regla práctica: Empieza siempre con prompting. Si no funciona después de iterar, considera RAG. Fine-tuning solo si hay razones muy concretas.

El **90% de los proyectos reales** se resuelven con prompting bien hecho.

<> Anatomía de un Buen Prompt — Componentes

1. **Rol** — "Eres un analista de producto..."
2. **Contexto** — los datos relevantes
3. **Tarea** — qué debe hacer exactamente
4. **Formato** — cómo debe responder
5. **Restricciones** — límites y condiciones

<> Anatomía de un Buen Prompt — Ejemplo

```
Eres un analista de atención al cliente
para una tienda de electrónica.

Review del cliente:
{review}

Clasifica esta review como: positiva,
negativa o mixta.

Responde en JSON con los campos:
"clasificacion" y "razon" (max 30 palabras).
```

<> Ecosistema Actual: APIs en la nube

- Sin infraestructura propia
- Últimos modelos disponibles inmediatamente
- Pago por token consumido
- Datos viajan a servidores de terceros
- Dependencia de proveedor y de precios

Ejemplos: OpenAI API, Gemini API, Claude API

<> Ecosistema Actual: Modelos open-source locales

- Se ejecutan en tu propio hardware (GPU)
- Control total de los datos
- Coste fijo (hardware + energía)
- Modelos menos capaces que los frontier
- Requieren expertise técnico

Ejemplos: Llama 3, Mistral, Phi-3, Qwen 3

Tendencia 2025–2026: Modelos cada vez más pequeños y eficientes permiten ejecutar capacidades significativas en hardware convencional — o incluso on-device (móvil, edge).

<> Costes de las APIs: La Aritmética Importa

Cómo se cobra

- Precio por **millón de tokens** (input + output)
- 1 token ≈ ¾ de palabra en inglés
- El coste escala linealmente con el volumen

Ejemplo: clasificar 100.000 reviews

- ~150 tokens de prompt por review → 15M tokens
- ~50 tokens de output por review → 5M tokens
- Total: 20M tokens

A $0.15/M tokens → **$3 USD**

A $15/M tokens → **$300 USD**

<> Costes de las APIs: Factores de riesgo

Factores que disparan los costes

- Contexto largo (documentos enteros en el prompt)
- Muchas iteraciones de desarrollo
- Modelo grande para tareas simples
- Sin rate limiting ni caching

Un sistema en producción sin gestión de costes puede generar facturas inesperadas de miles de euros en días.

<> ⚠️ Sección Crítica

El coste real de la IA generativa

<> El Consumo Energético de la IA

Una pregunta a ChatGPT consume ~10× más energía que una búsqueda en Google.

Datos documentados (2024–2025)

- Los data centers de IA representan el 1–2% del consumo eléctrico global y la cifra crece
- Microsoft, Google y Amazon han reactivado centrales nucleares para alimentar sus CPDs
- Se proyecta que el consumo de IA iguale el de toda la aviación comercial en 2030

<> El Consumo Energético de la IA — El agua también importa

Los data centers usan refrigeración por agua.

Entrenar GPT-3 requirió ~700.000 litros de agua.
Una conversación larga consume ~500 ml.

Esta escala de consumo es difícilmente compatible con los compromisos climáticos de las mismas empresas que venden estos servicios.

<> "Token Poverty": La Brecha de Acceso

¿Quién puede permitirse IA a escala?

- Grandes tecnológicas: presupuestos ilimitados
- Startups bien financiadas
- Investigadores en universidades ricas (créditos de proveedores)
- Empresas medianas: acceso manejable con uso selectivo

La promesa de "democratización"

ChatGPT gratuito, Gemini gratuito, GitHub Copilot...

<> "Token Poverty": La Brecha de Acceso

¿Quién queda fuera en escala industrial?

- Pymes en economías emergentes
- Investigadores en el Sur Global
- Periodistas, ONGs, sector público
- Cualquiera que necesite más de los límites del plan gratuito

La "democratización" real del acceso individual es cierta. La concentración del poder de **entrenar y controlar** estos modelos es un fenómeno diferente.

<> Concentración del Poder: Los Chips

Las GPUs son el cuello de botella de toda la industria

- NVIDIA controla >80% del mercado de chips para IA
- El H100 costaba $30.000–40.000 USD por unidad en 2024
- Tiempos de espera de 6–12 meses durante el boom de 2023–2024
- Las restricciones de exportación de EE.UU. hacia China crean tensiones geopolíticas activas

Consecuencia directa

Solo 5–6 organizaciones en el mundo pueden entrenar modelos frontier: Anthropic, OpenAI, Google, Meta, Microsoft, xAI

El resto del mundo **consume** lo que ellos producen bajo sus condiciones.

<> Concentración del Poder: Implicaciones

No es alarmismo: es estructura de mercado. Entender esto cambia cómo evaluamos la "soberanía tecnológica" de una empresa o un país.

<> Riesgos: Alucinaciones

El modelo genera información falsa con total confianza

¿Por qué ocurre?

- Predice tokens probables, no verdades verificadas
- No tiene mecanismo para "saber que no sabe"
- El entrenamiento penaliza la incertidumbre
- Confunde correlaciones estadísticas con hechos

Casos documentados

- Citas bibliográficas inventadas en papers
- Jurisprudencia falsa en escritos de abogados
- Estadísticas inventadas con decimales precisos

<> Riesgos: Alucinaciones — Mitigaciones

Mitigaciones

- RAG (conectar a fuentes verificadas)
- Pedir al modelo que cite fuentes
- Evaluación humana en decisiones críticas
- Nunca usar LLMs como fuente de verificación factual

Un sistema que alucina con confianza es más peligroso que uno que dice "no sé".

<> Riesgos: Sesgos y Privacidad

Sesgos

- Heredados del texto de entrenamiento
- Sobre-representación de contenido en inglés y occidental
- Sesgos de género, raza y cultura difíciles de eliminar
- Difíciles de auditar completamente

Privacidad de datos

- Los datos enviados a APIs de terceros pueden usarse para entrenamiento (leer los ToS)
- Riesgo de exponer información confidencial involuntariamente
- GDPR: datos de ciudadanos europeos procesados fuera de la UE

<> Riesgos: Regulación — EU AI Act (en vigor 2025)

Riesgo	Categoría	Obligación
Biometría masiva	Prohibido	—
Crédito, empleo	Alto riesgo	Auditoría
Chatbots	Riesgo limitado	Transparencia
Filtros de spam	Mínimo	Sin restricción

Los LLMs que toman decisiones con impacto en personas entran en la categoría de **alto riesgo**.

<> Práctica Guiada

`practica_prompts.ipynb`

Ejercicios

1. Conectarse a Gemini API
2. Clasificar una review (positiva / negativa / mixta)
3. Extraer información estructurada (producto, quejas, elogios)
4. Mejorar el prompt con rol, formato y restricciones
5. Comparar 3 reviews del mismo producto

Tiempo estimado: 10 min guiados + experimentación libre

<> Práctica Guiada — Cómo obtener la API key

1. Ir a aistudio.google.com
2. Crear cuenta Google
3. Get API Key → Create API key
4. En Colab: panel de Secrets → `GOOGLE_API_KEY`

Gemini 1.5 Flash gratuito: 15 requests/minuto y 1 millón de tokens/día. Suficiente para toda la práctica.

Sesión 1 — Recapitulación

Lo que hemos visto:

- Los LLMs son predictores de tokens entrenados a escala — no razonadores
- El prompting resuelve el 90% de los casos de uso reales
- Los embeddings permiten búsqueda y comparación semántica
- El coste real (energía, GPUs, concentración) forma parte del análisis de viabilidad

Próxima sesión:

Diseño de soluciones GenAI — RAG, casos de uso por función de negocio, Build vs. Buy

¿Preguntas?